

On-Device AI Systems for Backend Engineers

A from-scratch, wide-ranging working reference built while learning this space — starting from transformer/ML fundamentals, through the systems engineering of running inference locally on constrained hardware, back out to the general backend, OS/hardware, and engineering-process foundations all of it rests on.

Compiled while learning the space, mostly to have one place to keep coming back to

Not exhaustive — meant to be revised as I learn more

Sources: engine docs (ONNX Runtime, Core ML), llama.cpp internals, quantization papers (GPTQ/AWQ/SmoothQuant), general OS/scheduling theory, distributed-systems fundamentals

CONTENTS

I ML & Transformer Foundations (A1–A5)	IV OS, Hardware & Mobile Specifics (D1–D4)
II On-Device Systems Engineering (B1–B17)	V Engineering Practice & Process (E1–E3)
III Backend & Distributed Systems (C1–C5)	REF Glossary / Quick Reference

PART I

ML & Transformer Foundations

The modeling-side basics worth having solid before reasoning about how to run any of it efficiently on constrained hardware — architecture, tokenization, positional encoding, the training pipeline that produces a checkpoint, and compression techniques that sit alongside quantization.

-
- A1 Transformer Architecture From Scratch
 - A2 Tokenization Algorithms
 - A3 Positional Encoding Schemes
 - A4 The Training Lifecycle: Pretraining, SFT, RLHF/DPO
 - A5 Model Compression Beyond Quantization

Transformer Architecture From Scratch

§ The core computation

Every transformer block does the same two things in sequence: self-attention (let each token look at other tokens and pull in relevant context), then a position-wise feedforward network (transform each token's representation independently). Both are wrapped in residual connections and normalization, stacked N times.

§ Self-attention, mechanically

Each token's embedding is projected into three vectors: **query (Q)**, **key (K)**, and **value (V)**. A token's query is compared against every other token's key (dot product, scaled by `1/sqrt(head_dim)` to keep gradients well-behaved) to produce attention scores, which are softmaxed into weights and used to compute a weighted sum of value vectors. That weighted sum is the token's new, context-aware representation.

§ Why multiple heads

Splitting Q/K/V into several smaller "heads" that each attend independently (then concatenating the results) lets different heads specialize — one might track syntactic structure, another long-range coreference. This is also precisely the dimension that MQA/GQA (§B4) compress by sharing K/V projections across heads.

§ Feedforward block and normalization

After attention, each token independently passes through a small two-layer MLP (usually expanding to 4x the model dimension and back), which is where most of a transformer's parameter count actually lives. Layer normalization is applied either before each sub-block ("pre-norm," the modern default — more stable training at depth) or after ("post-norm," the original design). Residual connections around both the attention and feedforward sub-blocks are what make very deep stacks trainable at all.

WHY IT MATTERS DOWNSTREAM

Every optimization discussed later — quantizing weights, fusing operators, sizing the KV cache — is an optimization of this exact structure. Knowing which part of the block a technique touches (attention projections vs. feedforward weights vs. the cache) makes it much easier to reason about a new technique on sight.

Tokenization Algorithms

A tokenizer turns raw text into the integer IDs a model actually consumes. Subword tokenization is the near-universal choice because it handles rare/unseen words gracefully (falling back to smaller pieces) without a combinatorial explosion of a whole-word vocabulary.

Algorithm	How it builds a vocab	Used by
BPE (byte-pair encoding)	Iteratively merges the most frequent adjacent symbol pair, starting from individual bytes/characters	GPT-family models
WordPiece	Similar iterative merging, but chooses merges that maximize likelihood of the training corpus rather than raw frequency	BERT-family models
SentencePiece (Unigram)	Starts from a large candidate vocab and prunes down, keeping the subset that best preserves corpus likelihood; treats input as a raw stream (no whitespace pre-splitting), which handles non-whitespace-segmented languages better	Many multilingual models

RELEVANT ON-DEVICE

Tokenizer choice affects effective context length in a way that's easy to miss: a tokenizer with worse compression for a given language means more tokens (and more KV cache, §B7) for the same amount of actual text.

Positional Encoding Schemes

Attention itself has no inherent notion of token order — position has to be injected explicitly.

Scheme	Mechanism	Notes
Absolute sinusoidal	Fixed sine/cosine functions added to embeddings at each position	Original transformer design; doesn't extrapolate well beyond trained length
Learned absolute	A trainable embedding per position index	Simple, but hard-capped at the max position seen during training
RoPE (rotary)	Rotates Q/K vectors by an angle proportional to position before the attention dot product	Encodes <i>relative</i> position naturally; the dominant choice in current open models, including most edge-friendly ones
ALiBi	Adds a fixed, position-proportional penalty directly to attention scores instead of modifying Q/K	Extrapolates to longer sequences than trained on particularly well

WHY IT MATTERS DOWNSTREAM

RoPE-based models can sometimes have their effective context window extended post-training via interpolation/scaling tricks on the rotation angles — relevant if a device needs more context than a checkpoint officially ships with.

The Training Lifecycle: Pretraining, SFT, RLHF/DPO

§ Pretraining

Next-token prediction over a huge, broad text corpus. This is where most of the model's raw capability and world knowledge comes from — and where nearly all of the compute cost sits.

§ Supervised fine-tuning (SFT)

Fine-tuning the pretrained model on curated instruction/response pairs to shift it from "predicts plausible continuations" to "follows instructions in a helpful format."

§ Preference alignment: RLHF and DPO

RLHF trains a separate reward model from human preference comparisons, then optimizes the language model against that reward signal with reinforcement learning. DPO (direct preference optimization) achieves a similar effect more directly, optimizing the model against preference pairs without training a separate reward model or running full RL — simpler pipeline, has become a common lighter-weight alternative.

WHY IT MATTERS DOWNSTREAM

A model's alignment stage is part of what a quantization/compression pipeline can degrade — "quality" isn't just raw capability, it includes how well the aligned behavior survives compression. Evaluate compressed checkpoints against instruction-following behavior, not only base-model perplexity.

Model Compression Beyond Quantization

§ Pruning

Unstructured pruning zeroes out individual low-magnitude weights — high theoretical sparsity, but irregular sparsity patterns are hard for most edge hardware to exploit for an actual speedup. **Structured pruning** removes whole channels, heads, or layers — lower achievable sparsity, but the resulting model is a genuinely smaller dense model that any runtime can execute at full speed, which is usually the more practical choice for on-device deployment.

§ Knowledge distillation

Training a smaller "student" model to match a larger "teacher" model's output distribution (not just the training labels) — often recovers more quality at a given size than training that same small architecture from scratch on labels alone.

§ LoRA / QLoRA adapters

Instead of fine-tuning all of a model's weights, LoRA freezes the base model and trains a small pair of low-rank matrices injected alongside existing weight matrices — a few orders of magnitude fewer trainable parameters. QLoRA combines this with a quantized frozen base model, making on-device (or personalization-style) light fine-tuning tractable on hardware that could never fine-tune the full model. Relevant on-device specifically for any "personalizes to this user's data over time" feature without shipping a full new checkpoint per user.

SELF-CHECK

- ☐ For a given size budget, is pruning, distillation, or quantization (§B3) the better lever — and can more than one be combined?
- ☐ If a product needs per-user personalization, does a LoRA-adapter-per-user design fit the device's storage and load-time budget?

PART II

On-Device Systems Engineering

The core of this reference: what actually changes when a model has to run on one constrained machine instead of an elastic fleet — formats, quantization mechanics, hardware fallback, memory, streaming, scheduling, fault isolation, and the operational concerns (observability, power, security, deployment) that follow from those choices.

-
- B1 Why On-Device Is a Different Systems Problem
 - B2 Model Formats and Loading
 - B3 Quantization, In Depth
 - B4 Attention Variants and Memory Footprint
 - B5 Inference Engines and Runtime Selection
 - B6 Hardware Fallback Chains
 - B7 Memory Management and the KV Cache
 - B8 Streaming Architecture
 - B9 Concurrency and Scheduling
 - B10 Process Architecture and Fault Isolation
 - B11 Backend Fundamentals Applied to Inference
 - B12 Multi-Model Orchestration Under RAM
 - B13 Observability and Benchmarking
 - B14 Power and Thermal Awareness
 - B15 Security and Integrity
 - B16 Deployment, Versioning, Rollback
 - B17 Prior Art Worth Studying

Why On-Device Inference Is a Different Systems Problem

Server-side inference assumes near-infinite horizontal capacity: if a GPU box is busy, spin up another one. On-device inference assumes exactly one machine, with a RAM ceiling, a battery, a thermal envelope, and other apps competing for all of it at the same time. The interesting engineering isn't the model — it's everything around it that keeps a single, fragile process usable under those constraints.

Four things end up mattering more here than in typical backend work:

- **Failure is the default state, not the exception.** The inference worker will crash, run out of memory, or get starved of NPU access by another app. Design for that, don't patch around it later.
- **Latency budgets are tiny and visible.** A cloud API can hide 300ms behind a spinner. On-device, the user is watching tokens not appear.
- **There is no "just add capacity."** Every optimization is a tradeoff against a hard ceiling (RAM, thermal, battery) rather than a cost line item.
- **The environment is adversarial to consistency.** The same model on the same phone can behave differently minute to minute depending on thermal state, background app load, and battery mode — server fleets are homogenous by comparison.

§ A rough priority ordering

When tradeoffs conflict (and they will), a useful default ranking for a consumer-facing on-device assistant is: **correctness** > **availability (don't hard-crash the app)** > **latency-to-first-token** > **sustained throughput** > **raw model quality**. Model quality sits last not because it doesn't matter, but because a slightly worse answer delivered reliably and fast beats a better answer delivered inconsistently or with a frozen UI.

FRAME TO KEEP

Think of the on-device runtime the way you'd think of an embedded system that happens to run a neural network, not a stripped-down cloud server.

Model Formats and Loading

§ Why format choice is a systems decision, not a data-science one

The file format a model ships in determines how fast it can start, how safely it can be loaded from an untrusted source, and how much control the runtime has over partial/lazy loading. This is squarely backend territory even though the artifact is "a model."

Format	Loading model	Security	Notes
Raw PyTorch pickle (<code>.pt</code>)	Deserializes into memory directly; not naturally mmap-friendly	Unsafe for untrusted sources — pickle's <code>__reduce__</code> can execute arbitrary code on load	Fine for models you trained/built yourself; avoid for third-party downloads
safetensors	Flat header + raw tensor bytes; supports zero-copy / memory-mapped reads	Safe by construction — no executable deserialization path	Good default for anything sourced externally
GGUF (llama.cpp ecosystem)	Single file bundling weights, quantization metadata, tokenizer, and generation defaults; built for mmap	Safe, structured binary format	Convenient for edge deployment since one file is the whole deployable unit

§ Memory-mapped loading

Instead of reading an entire multi-gigabyte weight file into a heap buffer up front, `mmap` maps the file into the process's virtual address space and lets the OS page cache fault pages in on first access. Consequences worth internalizing:

- **"Loading" becomes mostly lazy** — startup latency drops because you're not paying for a full synchronous read before the first token can be produced.
- **The OS, not your process, owns residency decisions** under memory pressure — pages can be evicted and re-faulted transparently, which is a feature (automatic backpressure under system-wide memory pressure) and a risk (unpredictable latency spikes if a "hot" page gets evicted).
- **Multiple processes loading the same model file share physical pages** through the page cache — relevant if a warm-pool design ever runs two workers against the same checkpoint.

§ Lazy / layer-wise loading

Beyond mmap's implicit laziness, a runtime can explicitly stream in transformer layers in order, overlapping disk I/O for layer `N+1` with compute for layer `N` during the very first forward pass — shaving real time off cold-start latency rather than just deferring the cost.

§ Tokenizer loading

Easy to overlook: the tokenizer (SentencePiece/BPE merge tables, vocab) is small relative to weights but is on the critical path for *every single token*, not just at load time. It should load synchronously and stay resident — never subject to the same eviction logic as model weights.

GOTCHA

"The model loaded fast" and "the model is fast" are different claims when mmap is involved — a cold page fault storm during the first generation can show up as slow first-token latency even though the load call itself returned instantly.

Quantization, In Depth

§ PTQ vs QAT

Quantization-aware training (QAT) bakes quantization error into the training loop itself — best quality, but requires retraining, which is rarely an option for a third-party checkpoint. Post-training quantization (PTQ) compresses an already-trained model afterward using a small calibration dataset — the near-universal choice for edge deployment of open checkpoints.

§ Precision spectrum

Precision	Typical size vs fp16	Quality impact	Notes
fp16 / bf16	baseline	none	Default training/serving precision
int8	~50%	small, usually negligible with calibration	Safe default for most edge deployments
int4	~25%	noticeable on smaller models, less on larger ones	Needs a real PTQ method (GPTQ/AWQ), not naive rounding
sub-4-bit (2–3 bit)	~12–19%	meaningful quality cliff on most models below ~4 bits	Usually needs vector/codebook-based quantization, not scalar

§ Naive rounding vs real PTQ methods

Round-to-nearest (RTN) quantization — just picking a scale and rounding each weight — is the naive baseline and degrades fast below int8. Three methods worth actually understanding, since "which quantization method" is a real design decision with real tradeoffs:

- **GPTQ** — quantizes weights layer-by-layer, using second-order (curvature/ Hessian) information to choose roundings that minimize the reconstruction error of that layer's output, not just the weight values themselves. One-shot (no retraining), works reasonably well down to 4-bit.
- **AWQ (activation-aware weight quantization)** — observes that a small fraction of weight channels are disproportionately important based on the activation magnitudes that multiply them, and protects exactly those channels (via per-channel scaling) rather than quantizing everything uniformly. Often outperforms GPTQ at the same bit-width with a simpler calibration step.
- **SmoothQuant** — targets the harder problem of quantizing *activations* too (not just weights), which is where outlier channels historically broke int8 activation quantization. It migrates quantization difficulty from activations to weights via a per-channel smoothing factor computed from calibration data, making int8 weight+activation quantization viable without huge accuracy loss.

Method	Quantizes	Calibration needed	Typical use
RTN (naive)	weights only	none	Baseline / int8 only
GPTQ	weights only	small calibration set	4-bit weight-only, general purpose
AWQ	weights only	small calibration set	4-bit weight-only, often better quality/effort ratio than GPTQ
SmoothQuant	weights + activations	calibration set for activation ranges	int8 weight+activation, when activation-side speed matters

§ Granularity: per-tensor → per-channel → per-group

Per-tensor quantization uses one scale for an entire weight matrix — cheap, but wastes precision when values vary widely within that matrix. Per-channel uses a separate scale per output channel. Per-group (e.g., groups of 128 weights) sits between the two — most production 4-bit edge quantization defaults to group sizes in the 64–128 range as the practical sweet spot between accuracy and the extra scale-storage overhead.

§ Evaluating a quantized model properly

- Don't rely on perplexity alone — check downstream task accuracy and, for generative use, long-generation coherence (error compounds token over token in autoregressive decoding).
- Verify the quantization scheme is actually supported by the target runtime's kernels — a quantization format your calibration pipeline produces but your inference engine can't execute efficiently is a wasted step.
- Test on the actual target hardware, not just numerically in Python — some low-bit kernels have accelerator-specific rounding/behavior differences.

GOTCHA

Quantization error compounds across layers in autoregressive generation — a model can look fine on single-token quality checks and still drift over a long generation. Test on long sequences, not just short prompts.

Attention Variants and Memory Footprint

§ Why this belongs in a systems study guide

Attention variant is usually framed as a modeling choice, but on-device it's really a memory budgeting choice — it directly sets how large the KV cache grows per token, which in turn sets the maximum usable context length on a given device.

Variant	KV heads	Cache size vs MHA	Quality tradeoff
Multi-Head Attention (MHA)	one K/V per head	baseline	none — the original design
Multi-Query Attention (MQA)	single shared K/V head	$\sim 1/\text{num_heads}$	noticeable on some tasks; large cache savings
Grouped-Query Attention (GQA)	groups of heads share a K/V head	$\sim \text{num_groups}/\text{num_heads}$	middle ground; the common choice in current edge-friendly models

§ Sliding window attention

Rather than attending over the full history, the model only attends within a fixed-size recent window. This caps KV cache size independent of total context length — trading away long-range recall for a hard, predictable memory ceiling, which is often exactly the tradeoff an edge deployment wants.

§ Why it matters more here than server-side

A server can amortize memory across many concurrent requests (batching, paging, prefix sharing) so per-request KV footprint is one input among many to a scheduler. On-device, usually one request runs at a time, so the raw per-request KV cache size directly gates the maximum context length that device can support — there's no fleet-level averaging to hide behind.

SELF-CHECK

- ☐ For a target device's RAM budget, what's the actual max context length this attention variant allows, worked out from heads/layers/head-dim?
- ☐ Is the quality tradeoff of MQA/GQA acceptable for the task, or does long-range recall matter enough to keep more KV heads?

Inference Engines and Runtime Selection

The same model file can run through several different execution paths depending on platform. Picking one isn't a one-time decision — it's a per-device, sometimes per-request, decision.

§ The main players

- **ONNX Runtime** — cross-platform, pluggable "execution providers" let the same graph run on CPU, CUDA, DirectML, CoreML, or NNAPI backends without rewriting the model.
- **Core ML / Apple Neural Engine** — Apple's path to the ANE; best perf-per-watt on Apple silicon, but the ANE has real limits on supported ops, so unsupported layers silently fall back to GPU/CPU inside Core ML itself.
- **DirectML** — Microsoft's abstraction over whatever GPU/NPU is present on Windows; the practical fallback path when a vendor-specific NPU driver isn't available or fails.
- **llama.cpp / ggml-style runtimes** — CPU-first, memory-mapped model loading, heavily hand-optimized kernels. Often the most reliable fallback of last resort because it depends on the least amount of vendor-specific driver stack.

§ Graph-level optimizations worth knowing

- **Operator fusion** — combining sequences like matmul+bias+activation into a single kernel call to cut memory round-trips.
- **Constant folding** — precomputing anything that doesn't depend on runtime input.
- **Layout/format conversion** — reordering tensors into the layout a given accelerator wants, done once at load time rather than per-inference.

§ Speculative decoding

A small, fast "draft" model proposes several tokens ahead; the larger target model verifies them in a single forward pass and accepts the longest matching prefix, falling back to normal decoding at the first mismatch. Because decoding is typically **memory-bandwidth bound** (dominated by streaming weights through memory per step) rather than compute bound, this attacks the actual bottleneck: fewer sequential target-model steps for the same output, at the cost of running a second, smaller model alongside the first. Particularly attractive on-device because the "spare" compute headroom that speculative decoding needs is often sitting idle anyway between memory-bound steps.

§ Continuous / iteration-level batching – know the boundary

Server-side engines (batching many concurrent requests, admitting new ones mid-generation at the token-iteration level rather than waiting for a batch to fully finish) is largely a multi-tenant server technique. On-device, concurrency is usually capped low enough that this doesn't directly apply — but the underlying idea (schedule at fine granularity instead of coarse request granularity) shows up again in §09's scheduler design.

KEY INSIGHT

Don't pick "the best" engine once and hardcode it. Probe capability at runtime and keep the decision reversible per session, not baked in at build time.

Hardware Fallback Chains

§ The shape of the chain

A typical fallback order looks like: `NPU → GPU/Metal → CPU → (optional) cloud`. Cloud should never be a silent, automatic fallback — it changes the privacy and cost model of the request, so it belongs behind an explicit opt-in, not a "just in case" default.

§ Roofline thinking: compute-bound vs memory-bound phases

A single request has (at least) two distinct phases with different bottlenecks: **prefill** (processing the input prompt, highly parallel, closer to compute-bound) and **decode** (generating tokens one at a time, closer to memory-bandwidth-bound, since each step re-reads the full weight set to produce one token). The roofline model — performance capped by whichever of compute throughput or memory bandwidth is the binding constraint for the operation's arithmetic intensity — explains why optimizations that help prefill (raw FLOPs) don't always help decode (which wants faster memory access or fewer sequential steps, hence quantization and speculative decoding both helping decode specifically).

§ Circuit breaker pattern, applied to hardware

Treat each hardware backend as a circuit that can be *closed* (healthy, use it), *open* (recently failed, skip it and go to the next tier), or *half-open* (probe it occasionally to see if it recovered — a driver update or a thermal state change can bring a backend back without a restart).

```
state: CLOSED
  on inference_failure(backend) -> increment failure_count
  if failure_count > threshold -> state = OPEN, start cooldown timer

state: OPEN
  route all requests to next fallback tier
  on cooldown_expired -> state = HALF_OPEN

state: HALF_OPEN
  send one probe request to this backend
  on success -> state = CLOSED, failure_count = 0
  on failure -> state = OPEN, reset cooldown timer
```

§ Why runtime detection beats static assumption

Device specs on paper don't capture reality: a driver can be present but buggy, an NPU can be claimed by another foreground app, thermal throttling can silently degrade a backend mid-session. Detect capability by attempting a small real inference, not by reading a capability flag.

§ Thermal throttling as a moving target, not a one-time check

Sustained inference raises SoC temperature, which triggers firmware-level frequency scaling — visible externally as gradually decreasing tokens/sec over a long session even though nothing in the code changed. A benchmark taken in the first 10 seconds of a session is not representative of minute 5. The runtime should measure *sustained* throughput continuously and treat a sustained drop as its own signal — feeding into the same circuit-breaker logic used for outright failures, not just failures that throw an explicit error.

SELF-CHECK

- ☐ What happens if the fallback chain reaches the last tier and that fails too?
- ☐ How long should a cooldown be before probing a failed backend again?
- ☐ Does a fallback decision get made once per session or re-evaluated per request?

Memory Management and the KV Cache

§ What the KV cache actually is

During autoregressive generation, each new token attends over all previous tokens. Rather than recomputing key/value projections for the whole sequence at every step, the runtime caches them — the "KV cache." It grows linearly with sequence length, and scales with the number of layers and attention heads (see §04 for how attention variant changes this directly), which is why long-context generation is a memory problem before it's a compute problem.

§ Techniques to keep it under control

- **Windowing / truncation** — drop the oldest tokens' cache entries once a context window limit is hit (simple, lossy for long-range dependencies).
- **Quantized KV cache** — store cached keys/values at lower precision than the rest of the model; a high-leverage RAM saving since the cache can dwarf static weight memory at long context lengths.
- **Paged attention** — allocate KV cache in fixed-size pages/blocks rather than one contiguous growing buffer, with a per-sequence block table mapping logical positions to physical blocks. This enables non-contiguous allocation (less fragmentation) and, importantly, lets identical prefixes across requests (e.g., a shared system prompt) share the same physical pages instead of duplicating them — **prefix caching**.

§ Prefix caching in a bit more detail

If many requests share a common prefix (a system prompt, a few-shot template), computing and storing its KV cache once and reusing it across requests avoids redundant prefill compute entirely for that shared portion. On a single-user device this matters most for multi-turn conversation: the growing conversation history is itself a shared prefix between turn N and turn N+1, so retaining its KV cache between turns (rather than recomputing the whole conversation on every new message) is one of the highest-leverage latency optimizations available for chat-style interaction.

§ Model swapping under RAM pressure

When a device needs to serve more than one model and can't hold all of them resident, something has to give. Eviction policy options, in increasing sophistication:

Policy	How it decides	Weakness
LRU	Evict least-recently-used	A single one-off access can evict a genuinely "hot" model that's simply due for its next use soon
LRU-K	Tracks the last K access times, not just the most recent one	More resistant to one-off pollution; more bookkeeping
Cost-aware	Weights (model size freed) against (cost/time to reload) and recency together	Best fit for heterogeneous model sizes; needs reload-cost estimates kept up to date

§ Warm pool vs cold start

Keeping one warm standby worker ready means a crash or restart costs roughly a state handoff, instead of a full cold-load (disk read, buffer allocation, runtime init). The cost is holding that worker's memory reserved even while idle — a direct RAM-vs-recovery-latency tradeoff, usually worth it for the primary model, less clearly worth it for every model in a multi-model setup.

REMEMBER

At long context lengths, the KV cache — not the model weights — is often the dominant consumer of RAM. Budget for it explicitly instead of treating "model fits in RAM" as the whole story.

Streaming Architecture

§ Why streaming, not batching the response

Generation is inherently token-by-token. Buffering the full response before sending it back throws away the one advantage local inference has over a network round-trip: near-zero latency to the first visible token.

§ Transport options

- **Server-Sent Events (SSE)** — simple, one-directional, rides on standard HTTP, built-in reconnection semantics in most clients (`EventSource`).
- **WebSocket** — bidirectional, useful if the client needs to send mid-stream signals (cancel generation, adjust sampling temperature live) without opening a second connection; more infrastructure complexity (proxies/load balancers need explicit WS support).
- **Local IPC (unix domain sockets / named pipes)** — used internally between the supervisor/API layer and the inference worker; lower overhead than loopback HTTP, needs explicit message framing.

For a local loopback bridge between an internal worker and an external HTTP-facing API, SSE is usually sufficient — cancellation can ride as a small separate request/IPC signal rather than requiring a fully bidirectional stream.

§ A minimal event schema

```
// token chunk
{ "type": "token", "generation_id": "gen_8f2a", "index": 42, "text": "the" }

// error mid-stream
{ "type": "error", "generation_id": "gen_8f2a", "code": "WORKER_CRASHED", "recoverable": true }

// completion
{ "type": "done", "generation_id": "gen_8f2a", "total_tokens": 187, "finish_reason": "stop" }
```

§ Backpressure

If a client (or the UI thread rendering tokens) is slower than the model is generating, the transport needs an explicit policy: buffer up to a limit, drop and coalesce, or apply backpressure all the way back to the generation loop itself (pause decoding until the consumer catches up). Silently unbounded buffering is how a slow client becomes a memory leak.

§ A resumable-stream protocol sketch

Client tracks `last_received_index` per generation. On reconnect:

```
client -> server: resume(generation_id, from_index=last_received_index)

server:
  if generation still in progress or buffer has tokens from from_index onward:
    replay buffered tokens from from_index, then continue live streaming
  else:
    respond: cannot_resume(reason) -- client must decide: restart or give up
```

This requires the server to retain a small bounded ring buffer of recently emitted tokens per active generation — bounded so a permanently-disconnected client doesn't grow it unbounded.

SELF - CHECK

- ☐ What's the event schema for a token chunk vs. an error vs. a "generation complete" signal?
- ☐ If the SSE connection drops mid-stream, what does the client send to resume vs. restart?
- ☐ How large should the server-side replay buffer be, and what happens when it's exceeded?

Concurrency and Scheduling

§ Why an unbounded queue doesn't work here

Unlike a stateless web request, an inference request holds a chunk of memory and (often) a hardware context for its entire duration. Letting an unlimited number run concurrently doesn't just slow things down — it can exhaust RAM outright. A hard cap on concurrent inference "slots" is a correctness requirement, not just a performance tweak.

§ Admission control

Before accepting a new request, check current RAM headroom and queue depth, and reject or queue *proactively* — rather than accepting everything and discovering the problem as an out-of-memory crash mid-generation. A clear, fast rejection at admission time is strictly better than an unpredictable failure partway through.

§ Fair queuing beyond plain round robin

Plain round robin treats every queued request equally — structurally fair, but can't express priority (interactive foreground request vs. background prefetch). **Weighted round robin** assigns more "turns" to higher-priority classes. **Deficit round robin** goes further: each class accumulates a byte/token "deficit" quota per round that carries forward if unused, smoothing out bursty, variable-length work like token generation.

§ Priority aging

Even with weighted scheduling, a low-priority class can theoretically starve forever under sustained high-priority load. Priority aging raises a waiting request's effective priority the longer it waits, guaranteeing it eventually runs — a standard OS-scheduler technique that maps directly onto this problem.

§ Batch composition (where it applies)

If a device ever does run more than one generation concurrently, grouping requests with similar expected remaining length reduces the padding/fragmentation waste of forcing very different-length sequences through the same batch step — a lighter-weight, single-device echo of the batching heuristics that matter a lot more on a multi-tenant server.

KEY INSIGHT

The scheduler question is rarely "which algorithm is fastest" — it's "what fairness guarantee do I actually need," and picking the algorithm that provides exactly that guarantee.

Process Architecture and Fault Isolation

§ Split by failure expectation, not by function

The most useful way to decide process boundaries here isn't "what does this code do" but "how do I expect this code to fail." A model/tokenizer/accelerator call stack has a fundamentally higher crash rate than routing and auth logic — so it belongs in its own process, isolated so its crash can't take anything else down with it.

§ A workable three-process shape

Process	Responsibility	Expected failure rate
Supervisor	Spawns/monitors workers, owns the warm pool	Should be crash-free; if it dies, the whole thing dies
API/server layer	HTTP/SSE surface, auth, routing	Crash-free by design; supervised restart with backoff if it ever does
Inference worker	Model, tokenizer, accelerator calls	Expected to crash routinely — isolate and recover fast

§ Sandboxing the worker, defense in depth

Beyond crash isolation, the inference worker is a reasonable candidate for OS-level permission restriction: it typically needs no network access (a fully local generation path), and only needs filesystem access scoped to the model/tokenizer directory. Running it with reduced permissions limits blast radius if it's ever compromised — for instance via a maliciously crafted model file exploiting a parser bug — independent of the crash-recovery story.

§ Restart backoff

A naive "restart immediately on crash" policy causes restart storms if the crash is caused by something systemic (a corrupt cache file, a bad checkpoint) rather than transient. Exponential backoff with jitter — plus a circuit breaker that stops auto-restarting after N failures in a window and surfaces the failure instead — avoids turning one bad model file into a crash loop that pins a CPU core forever.

SELF - CHECK

- ☐ If the warm standby worker is itself mid-load when the active one crashes, what happens?
- ☐ What's the maximum acceptable time between "active worker crashed" and "a warm worker is serving requests"?
- ☐ What OS-level permissions does the worker process actually need, versus what it's given by default?

Backend Fundamentals, Applied to Inference

§ Idempotency in a streaming-generation context

Classic idempotency (same request twice, same effect) doesn't map cleanly onto something stateful and partially-delivered like a token stream. The useful reframing: idempotency here means a client can safely retry *without duplicating or corrupting output* — which requires:

- A stable request/generation ID the client and server both track
- The server tracking how many tokens have already been emitted for that ID
- A "resume from token N" path distinct from "start over" — retrying naively by re-sending the same prompt is not idempotent if generation has any randomness (temperature/sampling), since it can produce a different continuation than the one already partially streamed

§ An error taxonomy, not just "it failed"

Different failure categories deserve different client-visible behavior — collapsing them all into a generic error loses information the client needs to react correctly:

Category	Example	Client-visible behavior
ClientError	Malformed request, invalid params	Surface immediately, don't retry
TransientError	Backend probe failed once, worker restarting	Retry with backoff, transparent to user if fast enough
FatalError	Worker crashed and won't recover automatically	Surface to user, offer manual retry
ResourceExhausted	RAM budget or queue depth exceeded at admission	Fail fast with a clear message, don't silently queue indefinitely

§ Partial failure handling

If a stream drops mid-generation, the failure mode matters more than the failure itself: did the worker crash (state is gone, must restart generation), or did only the transport drop (worker is still generating, client just needs to reattach and resume, per §08)? Conflating these two leads to either wasted recomputation or corrupted resumed output.

§ Structured logging as a single-device tracing analog

No distributed tracing backend is needed to get most of the benefit of tracing: assign a request/generation ID and log structured, timestamped events — `accepted` → `dispatched_to_worker` → `first_token_emitted` → `completed/failed` — so a single device's local logs can reconstruct exactly where time went on any given request, after the fact, without needing to reproduce the failure live.

FRAME TO KEEP

Most of "backend fundamentals for on-device AI" isn't AI-specific at all — it's ordinary distributed-systems failure reasoning (idempotency, retries, partial failure, backoff, error taxonomies) applied to a system that happens to have a model in the middle of it.

Multi-Model Orchestration Under Constrained RAM

When a new request needs a model that isn't resident and there isn't headroom to load it, there are four levers, and a real system should be able to justify picking one over another rather than hardcoding a single response:

Option	When it fits	Cost
Unload another model (LRU / LRU-K / cost-aware, §07)	The evicted model isn't likely to be needed again soon	Next request for the evicted model pays full reload cost
Quantize further, on the fly	Some quality loss is acceptable for this request class	Extra compute at load time; irreversible for that session without reloading
Queue the request	The caller can tolerate added latency	Bounded wait; needs the scheduler/priority-aging logic from §09
Refuse with a clear error	None of the above is acceptable for this request's priority class	Worst UX, but honest and fast — better than silently degrading

§ Combining eviction with admission control

These two mechanisms (§07's eviction policy and §09's admission control) should share the same RAM-accounting source of truth. A request that would push memory over budget should trigger an eviction-vs-queue-vs-refuse decision at admission time, not be accepted and then handled reactively once the OOM already happened.

KEY INSIGHT

"Refuse clearly and fast" is an underrated option. A system that always tries to degrade gracefully can end up doing so unpredictably; sometimes a fast, explicit failure is the more honest design.

Observability and Benchmarking

§ Metrics that actually matter here

- **TTFT (time to first token)** — the single most user-visible latency number; dominated by load/prefill cost, not decode.
- **Sustained tokens/sec** — measured over the whole generation, not just the first few tokens, to catch thermal throttling (§06/§14).
- **Memory high-water mark** — peak RSS during a session, not just steady-state, since transient peaks (e.g., during a model swap) are what actually trigger OOM kills.
- **Crash rate per hour of active use** — the real-world analog of an SLO for a process explicitly expected to crash sometimes (§10).
- **Fallback-tier distribution** — what fraction of requests actually get served by each hardware tier in the field, which is the only honest way to know if the fallback chain is being exercised as designed or if it's silently falling back to CPU on most real devices.

§ Percentiles, not averages

A system that's fast on average but has a bad p99 (say, from thermal throttling after five minutes of sustained use) looks perfectly healthy in a naive average-based benchmark and fails exactly the users it needs to serve well — the ones on long sessions or weaker devices. Report p50/p95/p99 for every latency metric, not a single mean.

§ Device matrix testing

Benchmark across a deliberately chosen spread of low/mid/high-end devices, not just a developer's own high-end machine. Thermal throttling, RAM pressure, and driver quirks overwhelmingly show up on the low end of the matrix — a benchmark run only on flagship hardware will systematically miss the failure modes that matter most.

§ Cold vs warm, always both

Report cold-start (first request after app launch) and warm-path (subsequent requests) numbers separately. A benchmark that only measures the warm path hides exactly the cost a real user feels on first use of the day.

SELF - CHECK

- ☐ What's the p99 TTFT on the lowest-end device in the support matrix, not just the median on a flagship?
- ☐ Does sustained tokens/sec at minute 5 look meaningfully different from minute 0.5?

Power and Thermal Awareness

§ Proactive, not just reactive, throttling

Reacting to thermal throttling only after tokens/sec has already visibly dropped means the user has already felt the degradation.

Better: reduce concurrency or fall back a hardware tier *preemptively* when the OS reports a low-battery or power-saving state, rather than waiting for a hard failure signal.

§ Adaptive concurrency from a rolling throughput signal

Track a rolling window of tokens/sec during generation. A sustained drop below a threshold is itself a usable signal — feed it into the same circuit-breaker logic used for outright hardware failures (§06), rather than treating "still technically working, just slower" as a separate, unhandled case.

FRAME TO KEEP

Thermal and battery state are inputs to the scheduling and fallback decisions from §06/§09, not a separate subsystem bolted on afterward.

Security and Integrity

§ Model file integrity

Verify checksums or signatures for downloaded model weights before loading — a corrupted or tampered file should fail a verification step, not silently load and produce subtly wrong (or exploitable) behavior.

§ Why format choice (§02) is also a security decision

Pickle-based checkpoint formats can execute arbitrary code on load via `__reduce__` — a concrete security reason, not just a convenience one, to require safetensors/GGUF for anything sourced from outside a fully trusted, self-controlled pipeline.

§ Sandboxing, revisited

Combined with §10's process isolation: a worker that only needs local filesystem access to a scoped model directory and no network access should actually be denied broader permissions at the OS level, not merely "not use" them by convention. Defense in depth means the permission boundary holds even if the code itself has a bug.

Deployment, Versioning, and Rollback

§ Model versioning

Pin a specific model version per app release rather than always-fetch-latest. Allow over-the-air model updates, but keep the previous version cached and verified-working until the new one has actually been exercised successfully, not just downloaded.

§ Staged rollout

Roll a new model version out to a small percentage of the install base first, watching the observability metrics from §13 (crash rate, TTFT, fallback-tier distribution) before expanding — the same staged-rollout discipline used for any risky server-side deploy, applied to a model artifact instead of a binary.

§ Rollback

Because the previous version is kept resident/cached rather than discarded immediately, a rollback is a local switch, not a full re-download — important on-device where network conditions can't be assumed to be good at the exact moment a rollback is needed.

Prior Art Worth Studying

None of the process-isolation ideas here are novel to on-device AI — they're well-trodden ground in adjacent systems, and it's worth studying the originals directly rather than rediscovering the same lessons from scratch:

- **Multi-process browser architecture** — a single tab/renderer crashing doesn't take down the whole browser, because each is isolated in its own process from the browser's core process. Direct analog: isolate the crash-prone inference worker the same way a browser isolates a crash-prone renderer.
- **Editor extension host model** — third-party extensions run in a separate host process specifically so a misbehaving extension can hang or crash without freezing the main editor UI thread. Same shape as isolating the inference worker from the API/UI-facing process in §10.
- **llama.cpp** — CPU-first, heavily hand-optimized, mmap-based loading, minimal external dependencies. Worth studying specifically as the reference for "reliable fallback of last resort," since minimal dependency surface is exactly what makes a fallback tier trustworthy.

PATTERN TO NOTICE

Mature systems facing "should I isolate this component" converge on the same answer for the same underlying reason — blast-radius containment — across otherwise unrelated domains. That convergence is a stronger signal than any single system's specific implementation details.

PART III

General Backend & Distributed Systems Foundations

The broader backend-engineering theory that Part II's specific techniques are instances of — worth having independently solid, since it transfers to basically any system, not just this one.

- C1 Caching Theory, Broadly
- C2 Consistency Models and the CAP Theorem
- C3 Resilience Patterns Beyond Circuit Breakers
- C4 API Design Fundamentals
- C5 Core Data Structures Worth Having Cold

Caching Theory, Broadly

The KV cache (§B7) is one specific instance of a much more general problem: deciding what to keep, for how long, and what to do when a write happens.

Policy	Idea	Good fit when
LRU	Evict least-recently-used	Recency predicts future use
LFU	Evict least-frequently-used	Some items are durably "hot" regardless of recency
TTL-based	Expire after a fixed time regardless of use	Staleness itself is the risk, not memory pressure

§ Write strategies

- **Write-through** — write to cache and backing store synchronously; simple, consistent, slower writes.
- **Write-back** — write to cache first, flush to backing store later; fast writes, risk of loss if the process dies before flush.
- **Write-around** — write straight to the backing store, skip the cache; good when written data is unlikely to be read again soon.

CONNECT THE DOTS

Every model-swap eviction policy in §B7/§B12 is a cache-replacement policy with an unusually expensive "miss" cost (a multi-second reload) — which is exactly why cost-aware eviction earns its complexity there in a way it might not for a cheap in-memory cache.

Consistency Models and the CAP Theorem

CAP: under a network partition, a distributed system must choose between consistency (every read sees the latest write) and availability (every request gets a response). A single on-device system mostly sidesteps this — there's one machine, no partition to speak of — but the question reappears the moment a hybrid cloud-fallback path exists (§B6): if the device and cloud disagree on state (say, a synced conversation history), which one wins, and does the user notice?

§ Consistency models worth knowing by name

- **Strong consistency** — reads always reflect the latest committed write.
- **Eventual consistency** — reads may be stale temporarily, but converge given no new writes.
- **Causal consistency** — writes that are causally related are seen in order by everyone, even if unrelated writes aren't.

RELEVANT ON-DEVICE

A local-first design that occasionally syncs to cloud is implicitly choosing eventual consistency for that synced state — worth making that choice explicit rather than accidental.

Resilience Patterns Beyond Circuit Breakers

The hardware-fallback circuit breaker (§B6) is one member of a small, reusable family of resilience patterns worth knowing generally, since they show up at every layer of a system, not just the accelerator layer.

Pattern	Idea	Where else it applies here
Bulkhead	Isolate resource pools per dependency so one failing dependency can't exhaust resources needed by another	Keeping the API layer's thread/connection pool separate from anything touching the inference worker
Timeout	Bound how long any single call is allowed to take	Every IPC call to the worker needs one — an unresponsive worker shouldn't hang the API layer indefinitely
Retry with jitter	Retry failed calls with randomized backoff to avoid synchronized retry storms	Reconnecting a dropped SSE stream (§B8), restarting a crashed worker (§B10)
Rate limiting	Cap the rate of incoming requests/work admitted	Same role as admission control in §B9, generalized beyond just concurrency slots

KEY INSIGHT

None of these patterns are AI-specific. Recognizing "this is just a bulkhead" or "this is just a timeout" when looking at a new problem is what makes unfamiliar systems feel familiar fast.

API Design Fundamentals

§ REST vs gRPC vs GraphQL

Style	Shape	Fits well when
REST	Resource-oriented, HTTP verbs, usually JSON	Simple CRUD-shaped surfaces, wide client compatibility, human-readable debugging
gRPC	Strongly-typed RPC over HTTP/2, binary protobuf, native streaming	Internal service-to-service calls, or client-server pairs that can share a schema — good fit for a typed IPC-like contract
GraphQL	Client specifies exactly the fields it wants in one request	Aggregating many different resources per screen; less natural for a single streaming generation endpoint

§ Versioning and backward compatibility

Additive changes (new optional fields) are safe. Removing or renaming a field, or changing a field's meaning, is not — it breaks any client still running old code. A versioned event schema (§B8's token/error/done events, tagged with a version) lets the client and server evolve without a hard cutover.

Core Data Structures Worth Having Cold

The building blocks underneath most of Part II's designs are ordinary data structures — worth being able to reason about (and implement) from scratch, not just name-drop.

- **Ring buffer** — fixed-size circular buffer; the natural structure for the bounded token-replay buffer in §B8's resumable-stream design.
- **Hash map with an LRU eviction wrapper** — a doubly-linked list (for $O(1)$ recency reordering) plus a hash map (for $O(1)$ lookup) is the standard way to implement LRU caching in $O(1)$ per operation, underlying §B7/§B12's model-swap logic.
- **Priority queue / heap** — natural fit for a scheduler that needs "next highest-priority ready item" in less than linear scan time, relevant to §B9's fair queuing.
- **Bounded blocking queue** — the concurrency-primitive version of admission control (§B9): producers block or get rejected once the queue is full, rather than growing unboundedly.

SELF - CHECK

- ☐ Could I implement an $O(1)$ LRU cache from scratch on a whiteboard, not just describe it?
- ☐ What's the actual time complexity of the priority-aging scheduler in §B9 as queue depth grows?

PART IV

OS, Hardware & Mobile Platform Specifics

The platform-level realities underneath the abstractions used in Part II — what the OS and the silicon are actually doing, and what mobile platforms specifically constrain.

-
- D1 OS Process & Memory Fundamentals
 - D2 CPU / GPU / NPU Architecture Basics
 - D3 Mobile OS Power Management Constraints
 - D4 Compiler & Runtime Optimization Concepts

OS Process & Memory Fundamentals

§ Process vs thread

A process has its own isolated address space; a crash in one process cannot directly corrupt another's memory. Threads within a process share an address space — cheaper to create and communicate between, but a crash in one can take the whole process down. This is precisely why §B10 puts the inference worker in its own process rather than just a separate thread: the isolation guarantee only holds at the process boundary.

§ Virtual memory and paging

Every process sees a private virtual address space; the OS maps virtual pages to physical RAM (or to disk, if paged out) via page tables. This indirection is what makes mmap-based model loading (§B2) possible at all — a "loaded" model is really just virtual address space mapped to a file, with physical pages faulted in on demand.

§ Context switch cost

Switching the CPU from one process/thread to another has real overhead (saving/restoring registers, flushing parts of the cache, TLB effects) — part of why an excessive number of concurrent inference slots (§B9) can hurt throughput even before hitting a memory ceiling.

CPU / GPU / NPU Architecture Basics

§ Why these accelerators are architecturally different

Unit	Optimized for	Weak point
CPU	Low-latency, branchy, general-purpose sequential work	Low raw throughput for massively parallel matrix math
GPU	Massive data-parallel throughput (thousands of simple cores)	Higher fixed launch overhead per operation; wants large batches to stay efficient
NPU	Fixed-function matrix/convolution math at very high perf-per-watt	Narrow supported operation set — anything outside it falls back elsewhere (§B5)

§ SIMD and cache hierarchy

Modern CPUs execute the same instruction across multiple data elements at once (SIMD) — heavily hand-optimized CPU inference kernels (llama.cpp-style, §B17) lean on this directly. Memory sits in a hierarchy (L1/L2/L3 cache, then RAM), each level slower and larger than the last; the "memory-bandwidth-bound decode" observation from §B6 is really about data having to travel further down this hierarchy than the compute units can be fed.

Mobile OS Power Management Constraints

§ Android

Doze mode and App Standby buckets restrict background work (network access, wake locks, scheduled jobs) for apps that aren't actively in use, with restrictions tightening the longer an app has been idle. Any background prefetch or model-warming behavior (§B7's warm pool) has to be scheduled through the OS's job-scheduling APIs and respect these buckets — it can't simply run a background timer and assume it will fire on schedule.

§ iOS

Background execution is even more restricted by default — most work must either finish quickly after backgrounding or use specific background modes/background tasks the OS grants briefly. A long-running local inference worker generally needs to be tied to active foreground use, or explicitly justified background-task time, rather than assumed to keep running indefinitely.

CONNECT THE DOTS

§B14's "proactive throttling under power-saving state" isn't optional politeness — on both major mobile platforms, the OS itself will suspend or restrict a process that doesn't respect these constraints, regardless of what the app wants.

Compiler & Runtime Optimization Concepts

§ JIT vs AOT compilation

Ahead-of-time (AOT) compilation optimizes a model graph once, at build/deploy time, producing a binary tuned for a specific target. Just-in-time (JIT) compiles (or re-compiles) at first-run time on the actual device, which can specialize better to the exact hardware present but adds to cold-start latency — a direct tension with §B2's cold-start concerns.

§ Graph compilers

Tools in the XLA/TVM family take a model graph and apply target-specific optimization passes (operator fusion, memory layout selection, kernel auto-tuning) to produce code specialized for a given accelerator — the same operator-fusion idea from §B5, generalized into a whole compilation pipeline rather than a handful of manually-written fused kernels.

PART V

Engineering Practice & Process

How a system like this actually gets built and kept healthy over time, beyond the architecture itself — testing discipline, incident handling, and documentation.

- E1 Testing Strategy for ML-in-the-Loop Systems
- E2 Incident Response and Postmortems
- E3 Documentation and Runbooks

Testing Strategy for ML-in-the-Loop Systems

Layer	What it tests	Notes
Unit tests	Deterministic components — the scheduler, the circuit breaker state machine, the eviction policy — in isolation	These are ordinary code; test them like ordinary code
Golden-output tests	Given a fixed prompt and greedy (temperature=0) decoding, output matches a stored reference within tolerance	Catches regressions from a quantization or engine change, since exact bit-for-bit output can't be guaranteed across hardware
Load/soak testing	Sustained generation over long sessions	The only reliable way to actually observe thermal throttling (§B6/§B14) rather than assume it
Fuzzing malformed input	Truncated prompts, invalid encodings, adversarial tokenizer edge cases	The inference worker is expected to crash sometimes (§B10) — fuzzing is how you find out it crashes on <i>more</i> inputs than intended

Incident Response and Postmortems

Even a single-device system benefits from the same discipline a server fleet uses: when something goes wrong in the field (elevated crash rate, a bad model version rollout, §B16), having a clear process — detect via observability (§B13), mitigate (roll back, §B16), then write a blameless postmortem — turns an incident into a permanent improvement instead of a one-off fire drill.

§ Blameless, specifically

The goal of a postmortem is to find the systemic gap (a missing test, a missing metric, a fallback path that was never actually exercised) rather than to find who to blame — blame makes people hide problems next time instead of surfacing them.

Documentation and Runbooks

A process explicitly expected to crash routinely (§B10) needs a runbook, not just code: what the on-call/maintainer should check first when crash rate spikes, what each error taxonomy category (§B11) means operationally, and what the known failure modes from the edge-case catalog actually look like in logs.

FRAME TO KEEP

Documentation that only describes the happy path is documentation for a system that doesn't exist yet. The interesting parts to write down are the failure modes.

Glossary / Quick Reference

Attention (Q/K/V)

The core transformer mechanism: each token's query is compared against every other token's key to weight a sum of value vectors, producing a context-aware representation.

BPE / WordPiece / SentencePiece

Subword tokenization algorithms that build a vocabulary of frequent pieces so rare or unseen words can still be represented as a sequence of known subunits.

RoPE / ALiBi

Positional encoding schemes that inject token-order information into attention — RoPE via rotating Q/K vectors, ALiBi via a distance-proportional penalty added to attention scores.

RLHF / DPO

Preference-alignment training stages that shift a model toward human-preferred responses — RLHF via a learned reward model and RL, DPO via a more direct optimization against preference pairs.

Pruning (structured / unstructured)

Compression by removing weights (unstructured, irregular sparsity) or whole channels/heads/layers (structured, hardware-friendly) from a trained model.

Knowledge distillation

Training a smaller student model to match a larger teacher model's output distribution, rather than training on labels alone.

LoRA / QLoRA

Parameter-efficient fine-tuning via small trainable low-rank matrices injected alongside a frozen (optionally quantized) base model.

CAP theorem

Under a network partition, a distributed system must trade off consistency against availability.

Bulkhead

A resilience pattern that isolates resource pools per dependency so one failing dependency can't exhaust resources needed elsewhere.

Virtual memory / paging

The OS abstraction mapping a process's private address space to physical RAM (or disk), fetched on demand — the mechanism underneath mmap-based model loading.

SIMD

Single-instruction-multiple-data — executing the same operation across multiple data elements at once, heavily used by hand-optimized CPU inference kernels.

JIT / AOT compilation

Just-in-time (compile at first run, specializes to the exact device) vs. ahead-of-time (compile once at build time) approaches to producing optimized execution code for a model graph.

GGUF

Single-file model format (llama.cpp ecosystem) bundling weights, quantization metadata, and tokenizer; built for mmap loading.

safetensors

Flat header + raw tensor format supporting zero-copy loading, with no executable deserialization path (unlike pickle).

KV cache

Cached key/value attention projections from previous tokens, reused so each new token doesn't recompute attention over the whole sequence from scratch.

Quantization

Reducing numeric precision of model weights/activations (e.g. fp16 → int8/int4) to cut memory and often speed up inference, at some quality cost.

GPTQ

A one-shot post-training quantization method using layer-wise second-order error minimization; effective down to 4-bit weight-only quantization.

AWQ

Activation-aware weight quantization; protects the small set of weight channels that activations show are disproportionately important, rather than quantizing uniformly.

SmoothQuant

A technique that migrates quantization difficulty from activations to weights via a per-channel smoothing factor, enabling viable int8 activation quantization.

MQA / GQA

Multi-Query / Grouped-Query Attention — variants where multiple attention heads share (or partially share) K/V projections, shrinking KV cache size at some quality cost.

Sliding window attention

Attention restricted to a fixed recent window of tokens, capping KV cache size independent of total context length.

Execution provider

A pluggable backend (CPU, CUDA, CoreML, DirectML, etc.) that a runtime like ONNX Runtime can target without changing the model graph itself.

Speculative decoding

Using a small draft model to propose several tokens, verified in one pass by the larger target model, to reduce sequential decode steps.

Roofline model

A performance model where throughput is capped by whichever of compute or memory bandwidth is the binding constraint for a given operation's arithmetic intensity.

Circuit breaker

A pattern that stops routing traffic to a failing dependency for a cooldown period, then probes it before fully trusting it again — closed / open / half-open states.

Prefix caching

Reusing a previously computed KV cache for a shared prefix (system prompt, conversation history) across requests instead of recomputing it.

Paged attention

Allocating KV cache in fixed-size blocks with a per-sequence block table, instead of one contiguous growing buffer — reduces fragmentation and enables prefix sharing.

Backpressure

A mechanism for a slow consumer to signal a fast producer to pause, instead of the producer buffering unboundedly.

Deficit round robin

A fair-queuing scheduler where each traffic class accumulates a quota ("deficit") per round that carries forward if unused, smoothing out bursty, variable-sized work.

Priority aging

Gradually increasing a waiting task's effective priority the longer it waits, to guarantee it eventually runs even under sustained higher-priority load.

Admission control

Proactively checking resource headroom before accepting a request, rather than accepting and handling resource exhaustion reactively.

LRU-K

An eviction policy that tracks the last K access times per item instead of just the most recent one, resisting pollution from one-off accesses.

Warm pool

One or more pre-loaded, idle worker processes kept ready so a crash/restart is a fast handoff instead of a full cold start.

Idempotency (streaming context)

The property that retrying a request doesn't duplicate or corrupt already-delivered output — requires tracking generation progress by ID, not just re-sending the original request.

TTFT

Time to first token — the latency from request start to the first visible generated token, usually the most user-perceptible latency number.

end of notes – revise as understanding improves